

WELL-ARCHITECTED, MULTI-TIER AWS INFRASTRUCTURE

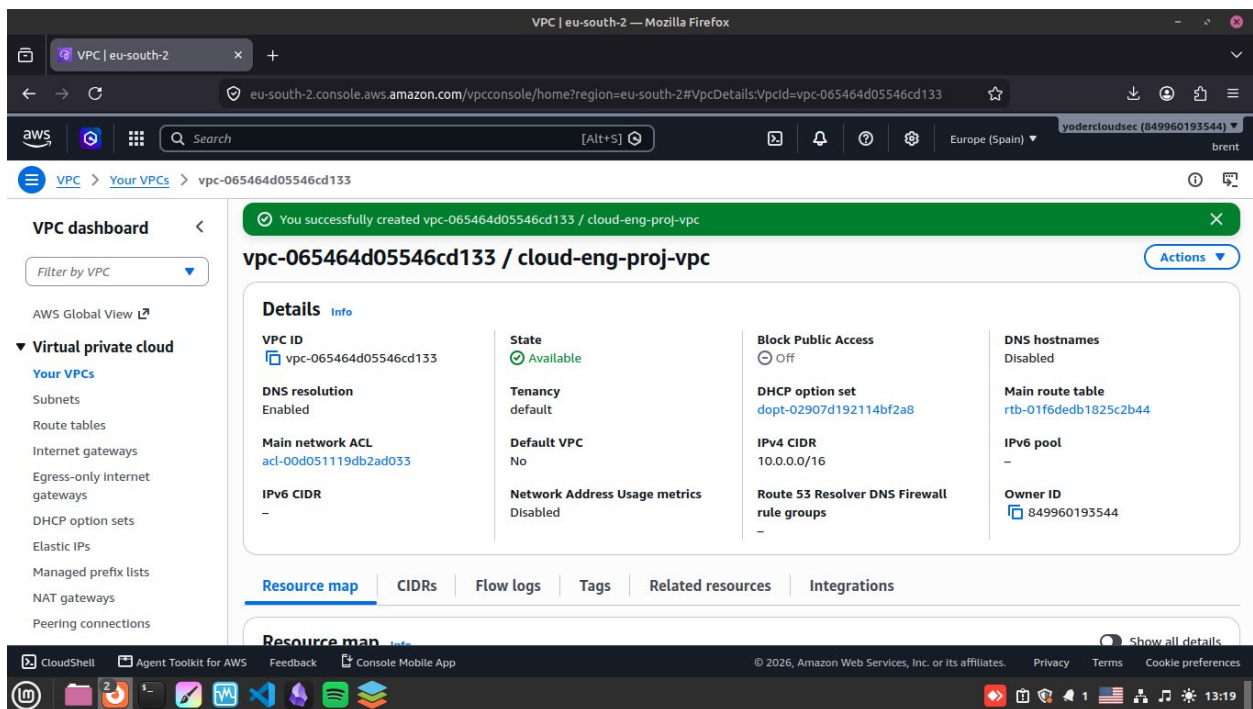
1. CREATE VPC

A VPC, or Virtual Private Cloud, is like your own little house within the cloud. This is where your resources are created and live. Everything is private by default and no one or nothing can come in or out of your VPC without your permission. I guess it's more like a dungeon than a house.

1. In the VPC Console, click Your VPCs, then 'Create VPC'
2. Choose 'VPC Only', name your VPC.
3. Choose an IPv4 CIDR block for your VPC.

I chose /16, which, technically is way too many (~65.5k available IPs in this range), but this leaves a lot of room for subnetting. In an enterprise or production environment, this would be great.

4. Create VPC.



The screenshot shows the AWS VPC console in the 'eu-south-2' region. A green notification bar at the top states: 'You successfully created vpc-065464d05546cd133 / cloud-eng-proj-vpc'. The main heading is 'vpc-065464d05546cd133 / cloud-eng-proj-vpc'. The 'Details' tab is active, displaying the following information:

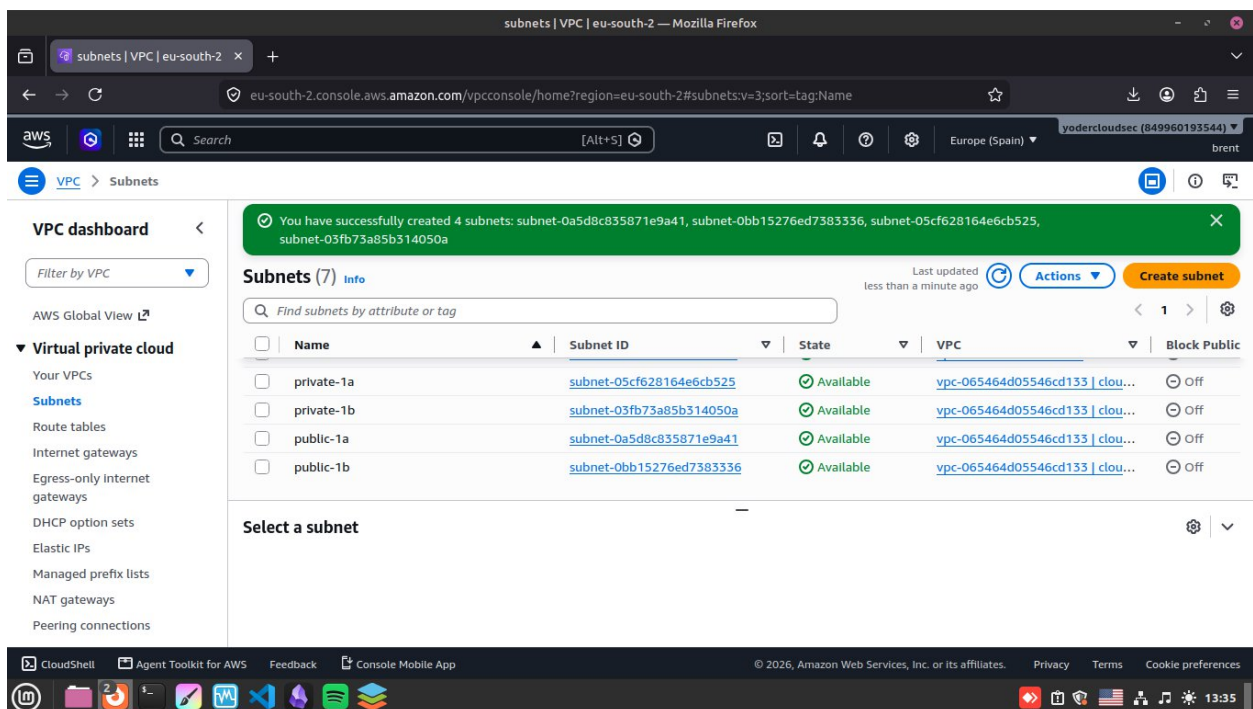
Details	State	Block Public Access	DNS hostnames
VPC ID vpc-065464d05546cd133	Available	Off	Disabled
DNS resolution Enabled	Tenancy default	DHCP option set dopt-02907d192114bf2a8	Main route table rtb-01f6dedb1825c2b44
Main network ACL acl-00d051119db2ad033	Default VPC No	IPv4 CIDR 10.0.0.0/16	IPv6 pool -
IPv6 CIDR -	Network Address Usage metrics Disabled	Route 53 Resolver DNS Firewall rule groups -	Owner ID 849960193544

Below the details, there are tabs for 'Resource map', 'CIDRs', 'Flow logs', 'Tags', 'Related resources', and 'Integrations'. The 'Resource map' tab is currently selected. The bottom of the screen shows the AWS console navigation bar with various service icons and the user's profile.

2. CREATE SUBNETS

If the VPC is your house, the subnets are the rooms of the house. A subnet allows you to logically divide your VPC into parts. Some parts will have internet access, some won't. Some will house databases, others will contain servers or other resources.

1. Under 'Subnets', click 'Create Subnet'
2. Under 'VPC ID' associate your subnet with the VPC we just made.
3. Name your subnets. We will make 4 of them, 2 public and 2 private. Naming them 'Public' and 'Private' will make your life easier later, so follow that norm. Make 1 private and 1 public subnet across 2 Availability Zones. Why? High-availability is the name of the game. This is achieved, in part, through Multi-AZ deployments.
4. Assign subnet CIDR blocks to each subnet. The size of your subnet must be smaller than the VPC CIDR block size we created earlier. The VPC is /16, so the subnets can be /17 to /32. You also cannot overlap subnet CIDR blocks within the same VPC.

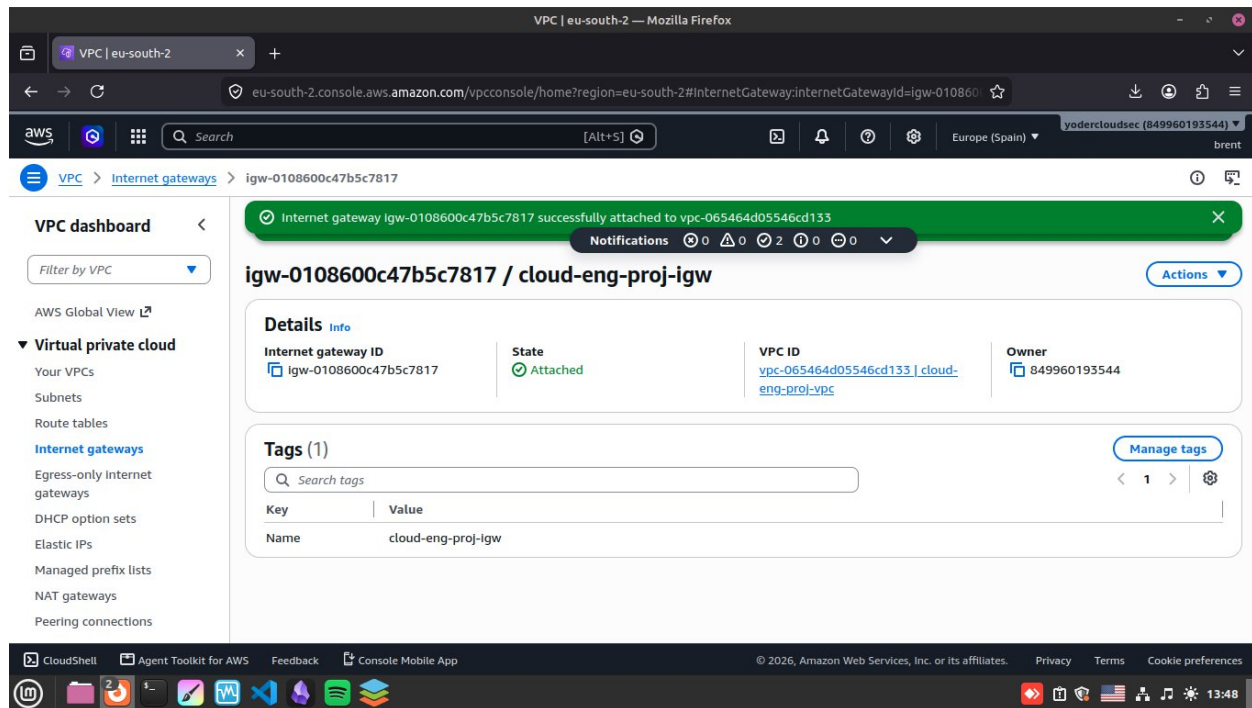


3. CREATE IGW

Resources inside subnets and VPCs can't naturally communicate with the internet. This is where an IGW or an Internet Gateway comes in. IGWs give internet connectivity to your resources inside the VPC. This could be allowing external users to access your resources, vice versa or two-way communication.

1. Under 'Internet Gateways', create and name your IGW.

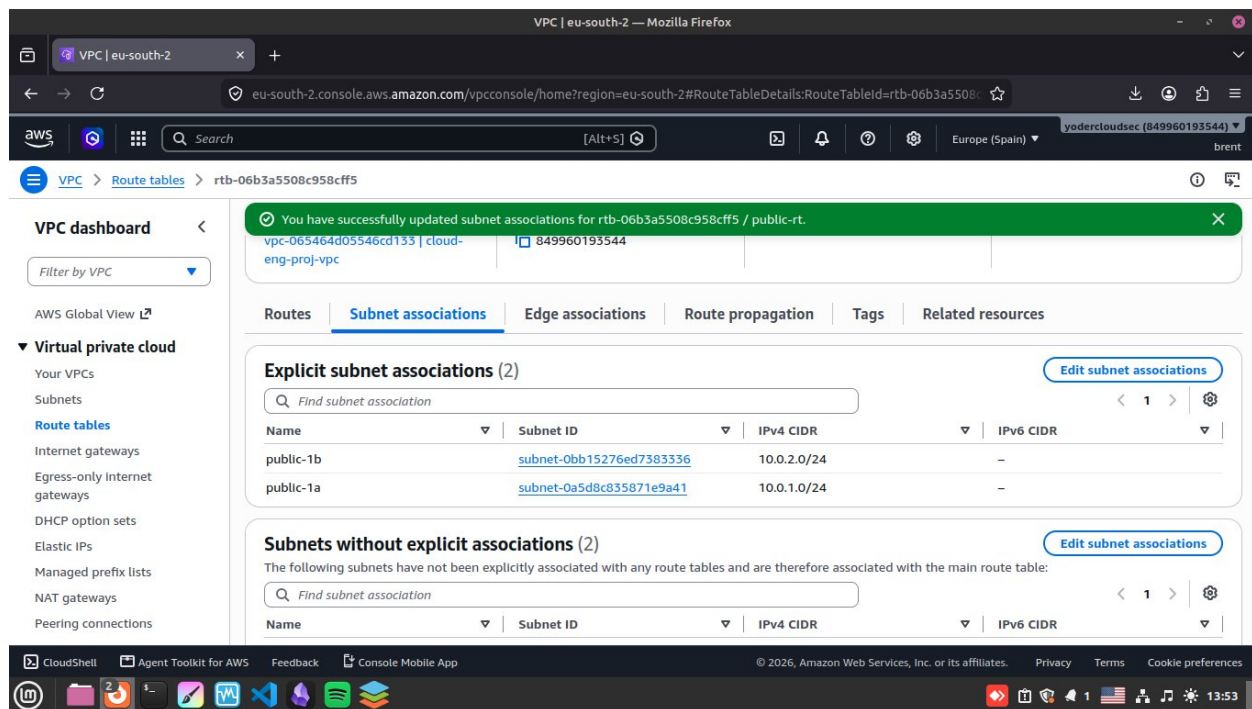
2. At the top right, under 'Actions', attach to VPC, then attach the IGW we just made to the VPC we just made. If you don't associate your IGW with a VPC, you'll never internet connectivity.



4. CREATE ROUTE TABLES

Wiring route tables is not fun, but an absolute necessity. As the name implies, route tables route govern where traffic is directed from subnets or gateways.

1. Under 'Route Tables', create and name yourself a Route Table. Since we will associate public instances with it, name it public-rt or something to denote that.
2. Click 'Edit Routes', 'Add Route'. Destination will be 0.0.0.0/0 (the entire internet) and select 'Internet Gateway'. Directly underneath, select the IGW we just created.
3. Click 'Subnet Associations' then 'Edit subnet associations'. Check the box for both public instances we created earlier and save. If subnets aren't explicitly associated, the routing rules fall under the main table rules and not the 'public-rt' table we just made. By default, the main table rules are private and therefore not able to reach the internet or be reached.
4. Repeat steps 1 and 3, but this time for a private route table, associating your private subnets. Skip step 2. Don't open them up to the internet, they're private. We will point the private route table at the NAT GW in the next few steps.



4.5. TROUBLESHOOTING

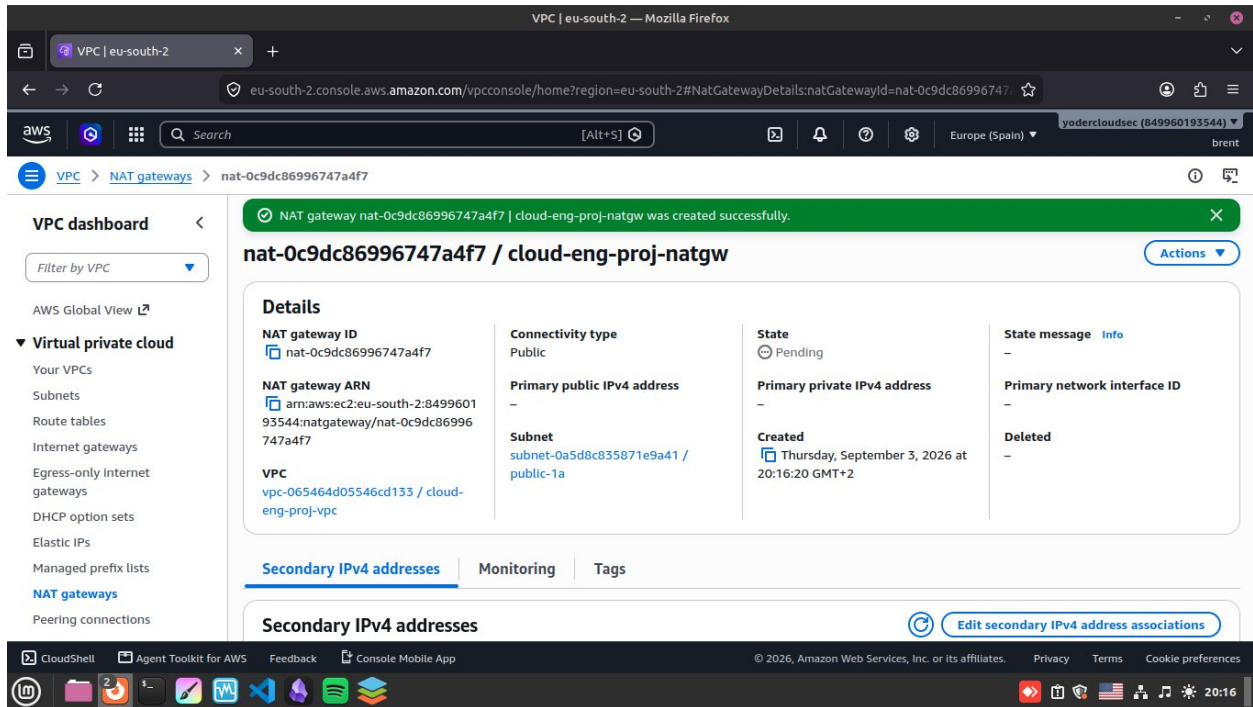
You're about to have a problem and you don't even know it yet. Ask me how I know. However, we will fix it before it becomes your problem. Your public subnets, even though they are associated with the IGW, need a public IP. AWS will auto-assign your subnets an IPv4. Otherwise the instances that will go into the public subnet later, won't have a public IPv4 address.

1. Subnets -> select public-1a -> Actions -> Edit subnet settings -> Auto-Assign IPv4
2. Repeat for public-1b

5. CREATE NAT GATEWAY

What if you only want your private EC2 instances/resources to only be able to connect outwardly to the internet when it's time to update or patch? This is what the NATGW is for. It allows outbound traffic, but doesn't allow the internet to directly access the subnet. It also allows for connectivity between other services in other VPCs and your on-premise networks.

1. Under 'NAT Gateways', create a NATGW. Name it and select 'Zonal Availability', choosing public-1a as the zone.
2. Allocate a new Elastic IP, save and continue.
3. Under 'Route Tables', edit the routes of the private route table. Add 0.0.0.0/0 with the destination being a NAT Gateway, selecting the NATGW we just made.



6. CREATE EC2

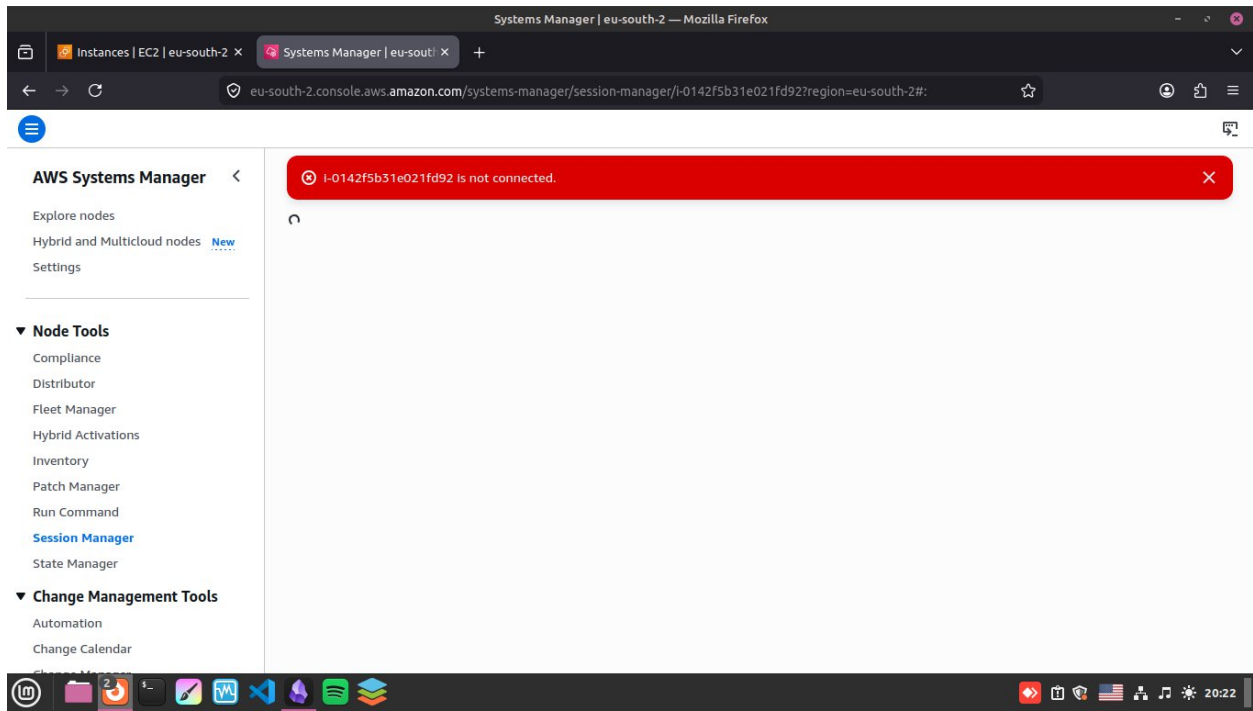
EC2 instances are the servers and virtual machines provided by AWS. There are plenty of varieties, some with pre-installed tools, others simply with an OS installed. We are going to use a pre-built AMI (Amazon Machine Image) of Linux for our purposes. It should be noted that different instance types serve different purposes.

1. In the EC2 console, launch an instance. Give it a name.
2. Under the AMI section, leave the default Amazon Linux selected, as well as the 't3.micro' default instance type.
3. Under Key Pair, select 'Proceed without a Key Pair'. This isn't necessarily a security best-practice, but it's also not the scope of this write up.
4. Allow AWS to create a Security Group for your instance.
5. In the Networking settings, select 'Edit' and select your VPC, then select 'private-1a' as the destination for where you will launch this EC2. Save then launch the instance.

7. CONNECT VIA SESSION MANAGER

There are two primary ways to connect to your newly created instance, SSH over port 22 and Session Manager. Why would I need to connect to an EC2 instance in a private subnet? Well for updating, among other things. Session Manager is safer from a security standpoint as it doesn't need port 22 open. Less open ports, less attack surface.

1. Still in the EC2 console, select your new EC2 instance and click 'Connect' at the top, then select 'Connect via Session Manager'.
2. It doesn't work.



7.5. TROUBLESHOOT 2: ELECTRIC BUGALOO

Why doesn't my EC2 instance connect over session manager? Well, it's due to IAM permissions. Not your user permissions, but Session Manager's. Session Manager works by having the SSM agent (which runs on the instance), reach out to Systems Manager to complete the command.

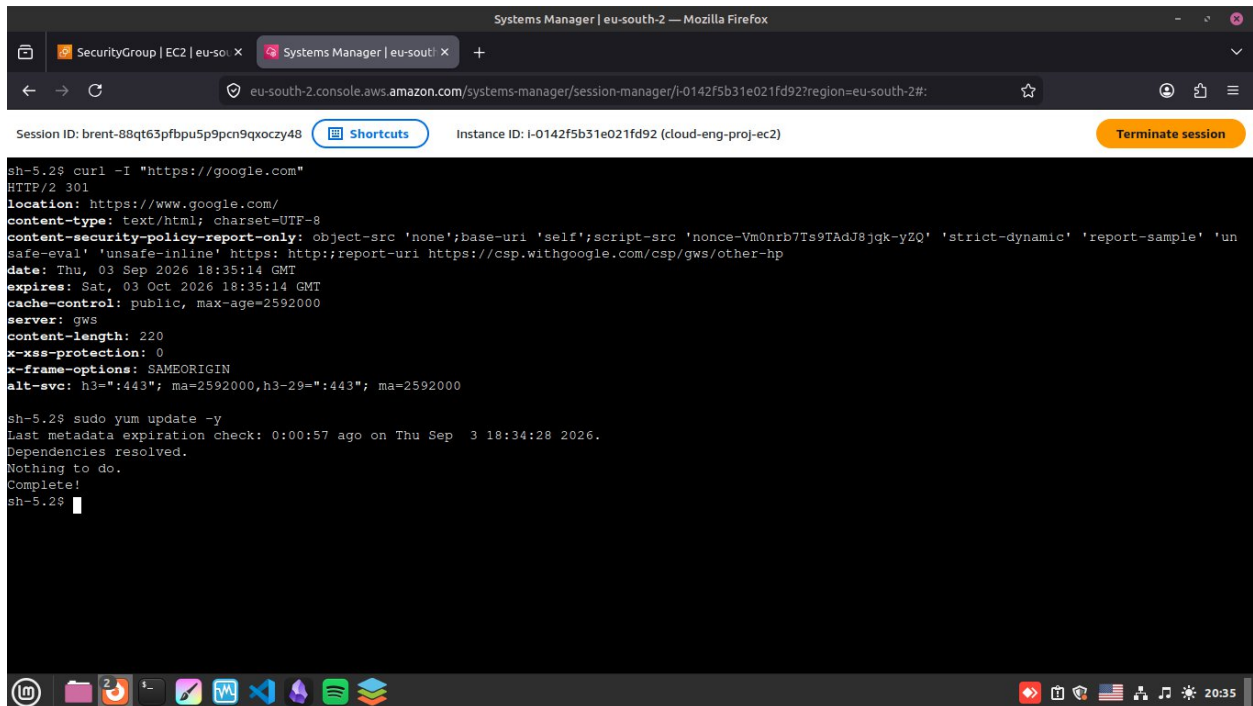
How do we fix this? By creating an IAM role for SSM and attaching it to our EC2.

1. Highlight your EC2 instance and under 'Actions' select 'Modify IAM Role'. Create a role.
2. Entity Type is AWS Service and Use Case is EC2.
3. In the policies section, search for 'AmazonSSMManagedInstanceCore' and select it.
4. Name your role whatever your little heart desires and attach it to your EC2 instance.

8. CONNECT VIA SSM

Now that we have that all sorted out, we can try connecting to our EC2 again via Sessions Manager.

1. Once connected, you'll see the typical Linux terminal. This doesn't necessarily guarantee outbound connectivity, so we have to try it.
2. Type 'sudo yum update'. Does it check for updates? That's a good sign.
3. Test a little further by typing 'curl -I https://google.com'. This should return the headers for Google.



```
sh-5.2$ curl -I "https://google.com"
HTTP/2 301
location: https://www.google.com/
content-type: text/html; charset=UTF-8
content-security-policy-report-only: object-src 'none';base-uri 'self';script-src 'nonce-Vm0nrb7Ts9TAdJ8jqk-yZQ' 'strict-dynamic' 'report-sample' 'un
safe-eval' 'unsafe-inline' https://report-uri https://csp.withgoogle.com/csp/gws/other-hp
date: Thu, 03 Sep 2026 18:35:14 GMT
expires: Sat, 03 Oct 2026 18:35:14 GMT
cache-control: public, max-age=2592000
server: gws
content-length: 220
x-xss-protection: 0
x-frame-options: SAMEORIGIN
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000

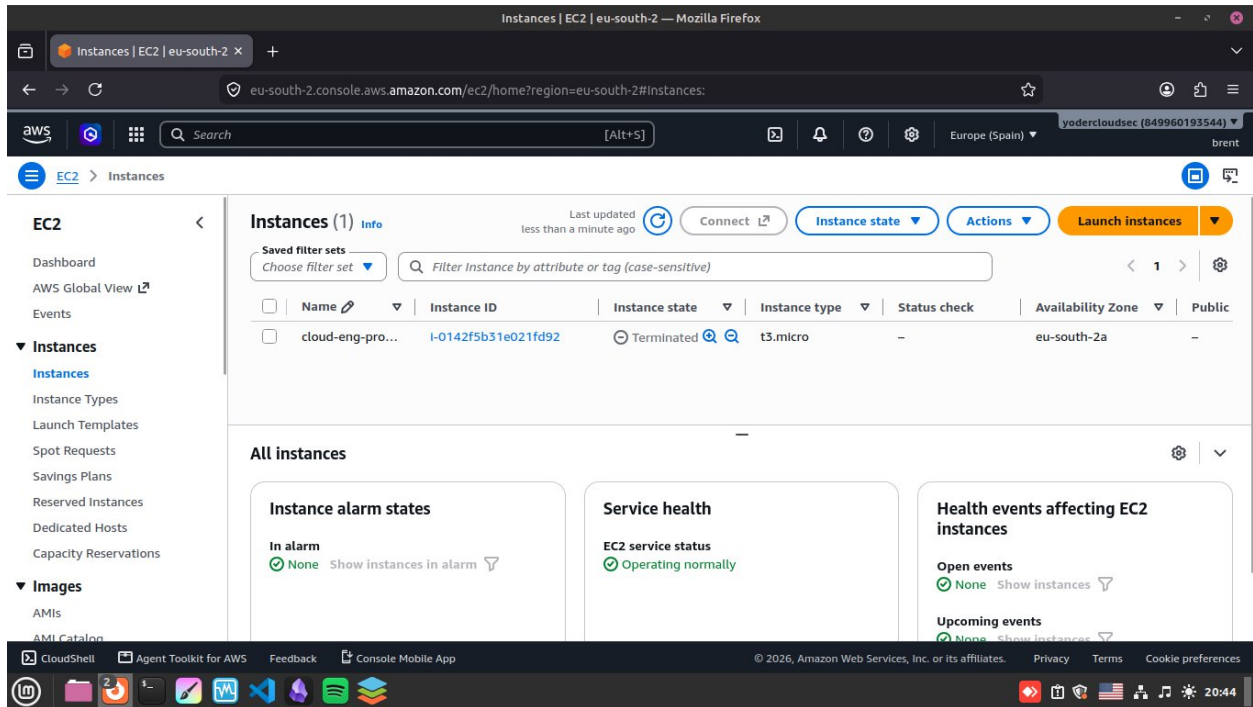
sh-5.2$ sudo yum update -y
Last metadata expiration check: 0:00:57 ago on Thu Sep 3 18:34:28 2026.
Dependencies resolved.
Nothing to do.
Complete!
sh-5.2$
```

If it works, and I'm sure it did, you just completed the first part of the tutorial. Let's tear down and delete the NATGW, EC2 Instance and the Elastic IP so as not to run up a bill.

9. MONEY SAVING TIPS

Let a NATGW run all night? In this economy? I think not. If you're no longer on the free plan for AWS, you're paying money, albeit a small amount per hour, for some of the things we just created. So, let's delete them if we aren't using them.

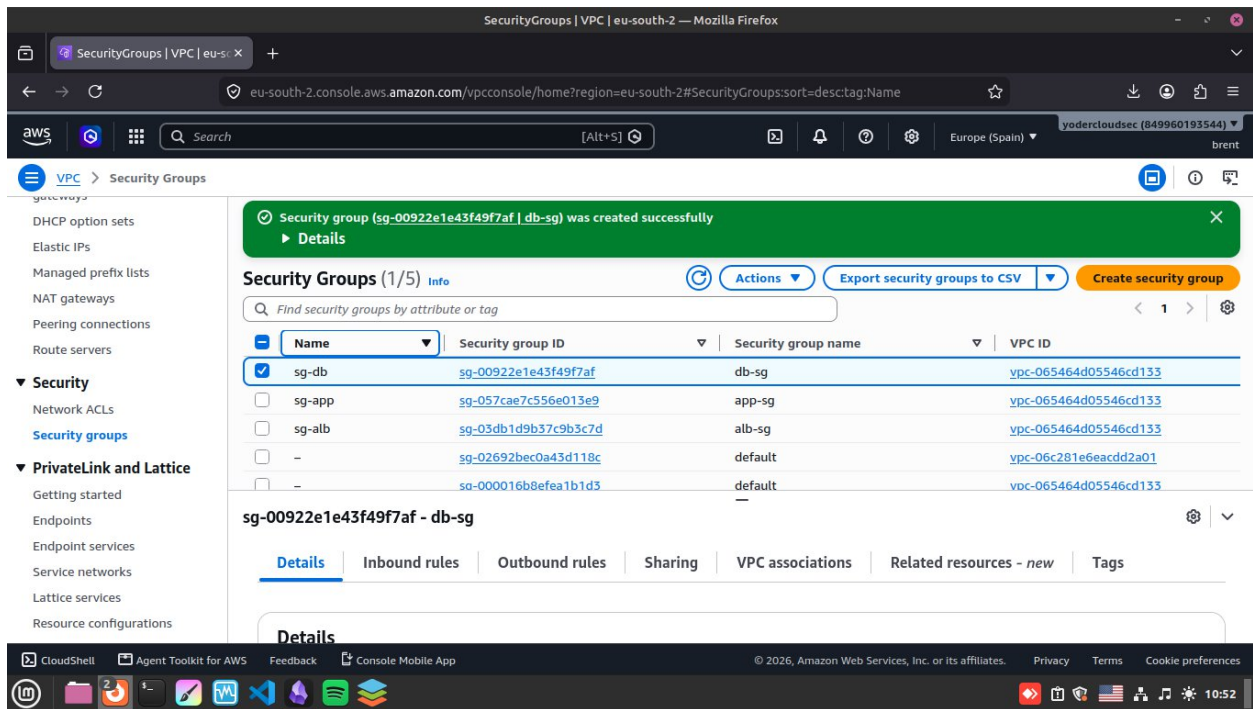
1. In the EC2 Console, select your EC2, under 'Actions', delete your instance.
2. In the VPC Console, under 'NAT Gateways', select your NATGW and delete it the same way.
3. Still in the VPC Console, under 'Elastic IPs', select the EIP that was associated with your now defunct NATGW and delete (Release) it as well. Just because your NATGW is gone, your EIP remains and you pay for it until you release it back into the wild.
4. Go back and double-check that your EC2 and NATGW have 'Deleted' or 'Terminated' under their 'Status' tab. The names of your EC2 and NATGW will remain for a little bit, but so long as the status is either deleted or terminated, you're not being charged.



10. SECURITY, SECURITY

Security Groups act as a firewall at the instance (EC2) level. Like a firewall, SGs can cause headaches and troubleshooting errors if they aren't properly set up. It's worth noting that SGs are Stateful, meaning they will allow outbound traffic somewhere once an inbound connection has been approved. Alas, we will be making 3 SGs, at 3 tiers, which all talk to each other. This type of SG-to-SG chaining is a good idea as far as secure architecture is concerned. Each SG will only talk to the tier/SG in front of it.

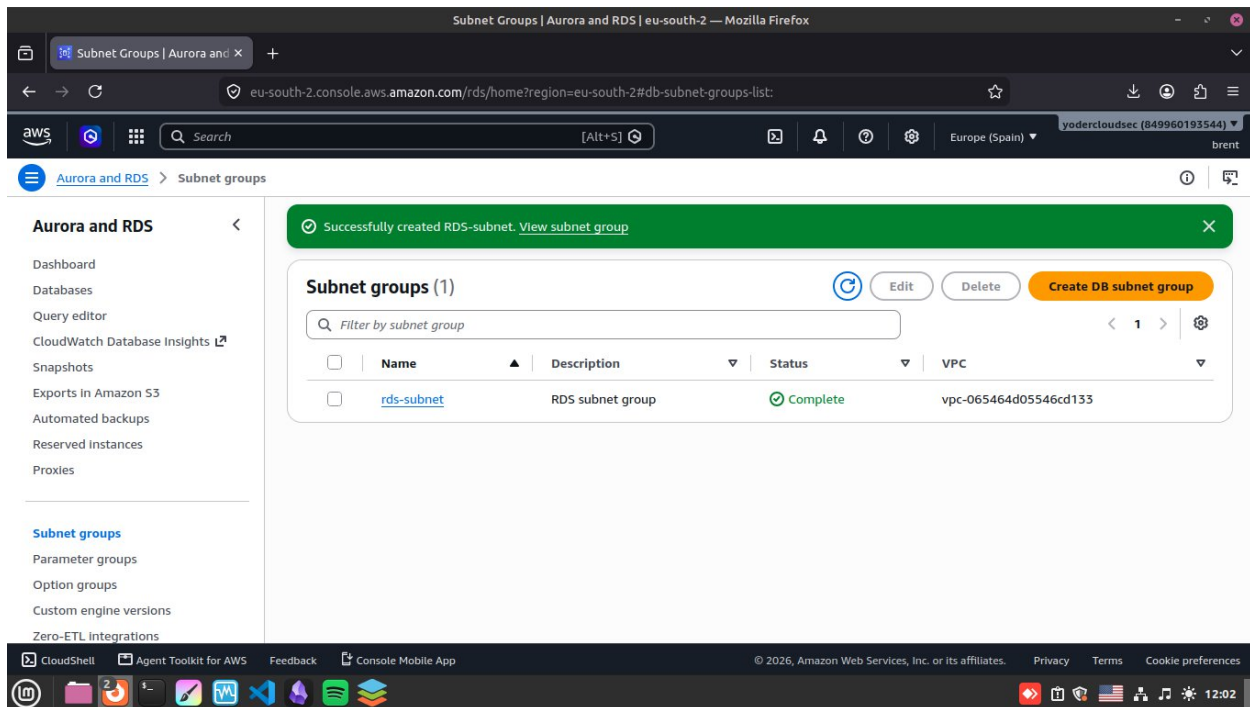
1. In the VPC Console, select 'Security Groups' and then create a Security Group
2. Name the first 'alb-sg' or something akin to it. Inbound traffic is HTTP, with the source being 0.0.0.0/0. This is the only SG that will talk to the internet directly. Save it and create another.
3. The second SG will be 'app-sg' or whatever. Inbound traffic is HTTP with the source being the alb-sg. This SG only accepts traffic from the app-sg. Save and create a third SG.
4. The last SG will be 'db-sg', if you please. Now, inbound traffic is MySQL and the source being app-sg. You guessed it, this SG only communicates with the SG above it, the application security group.



11. CREATE DB SUBNET GROUP

We're going to be setting up an RDS instance as our DB. RDS is a relational database (RD) service (S). Think MySQL, PostgreSQL, Oracle, etc. Rows and columns, basically. Anyway, it needs a subnet.

1. In the RDS/Aurora Console, select Subnet Groups and create one.
2. Give it a name and select your VPC, then the AZs your private subnets are in. Below that, select your two private subnets, 1a and 1b. This is a requirement for RDS failover to work: the group needs multi-AZ deployment even though we will only have a 1 AZ instance of RDS.
3. Create your RDS Subnet Group



12. CREATE RDS DATABASE

We're going to make a MySQL DB on RDS. Why RDS and not Aurora? A few reasons:

- RDS will allow you to run MySQL/PGSQL that you'd run anywhere and teach you vanilla MySQL/PGSQL skills that are cloud and environment agnostic. Aurora is essentially MySQL/PGSQL compatible, with a different storage layer and DB engine. Lots of cool features that become more useful at-scale and in lots of use cases, but overkill to use now, just for the sake of having a DB.

- \$ vs \$\$\$\$. RDS is cheap. Aurora isn't cheap, but the features like fast failover, read scaling, etc. make it worth paying for. Aurora's pricing model reflects that and assumes you've chosen it for those features and you have a need for those features.

1. In the RDS Console, select 'Databases' then create a full configuration DB. Select MySQL for the engine, not Aurora MySQL.
2. Select Single-AZ deployment. Multi-AZ and Multi-AZ Cluster are good for auto failover and would be the obvious choice for a production-level DB, but just to test tiered architecture, without real user traffic, it's an expensive and out-of-scope option.
3. Create a master user/password. You can choose to let AWS handle it through Secrets Manager, and in real life, you should, or you can do self-managed now.
4. Instance Type, select 'Burstable Class' then the 'db.t3.micro' instance type. This is the cheapest one you're going to find and more than sufficient for our purposes. Or select whatever is Free-Tier eligible if you have that option.

5. Select our VPC we made. DB Subnet Group will be the RDS DB subnet group we just made. Public Access set to 'No'. This is private property. The VPC SG will be the db-sg we made earlier.
6. You can turn off the Enhanced Monitoring if you want. You'd want EM in the real world.
7. Under 'Additional Configuration', name your DB whatever you want. Turn off the deletion protection as we are going to delete it after the tutorial is done. This is a bad idea in real life. Create your DB.
8. Once it's created, remember your username/password and click your instance. Scroll til you see the DNS name and copy it, you'll need it in a scooch.

13. CREATE EC2

We're going to launch an EC2 instance in 'private-1a'. This is the same process as before. Make the Security Group the 'sg-app' from before and attach the SSM role we also created earlier. This gives us secure access to the instance without opening port 22. Don't give it a public IP, it's a private instance.

14. NAT GW REARS ITS UGLY HEAD AGAIN

Make another Nat GW. Your EC2 instance needs it to reach outward to install the DB and SSM Agent Registration. It's as simple as creating it again and re-routing your 'private-rt' Route Table to it. Once the NAT GW is created, go to your 'private-rt' Route Table and change the route for the old NAT GW to the new. It'll say something along the lines of 'x Blackhole' or whatever until you do.

VPC | eu-south-2 — Mozilla Firefox

eu-south-2.console.aws.amazon.com/vpcconsole/home?region=eu-south-2#EditRoutes:RouteTableId=rtb-0fe9f9686a80be2a6

aws

VPC > Route tables > rtb-0fe9f9686a80be2a6 > Edit routes

Edit routes: rtb-0fe9f9686a80be2a6 / private-rt

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
0.0.0.0/0	NAT Gateway	-	No	CreateRoute

Add route

Cancel Preview Save changes

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

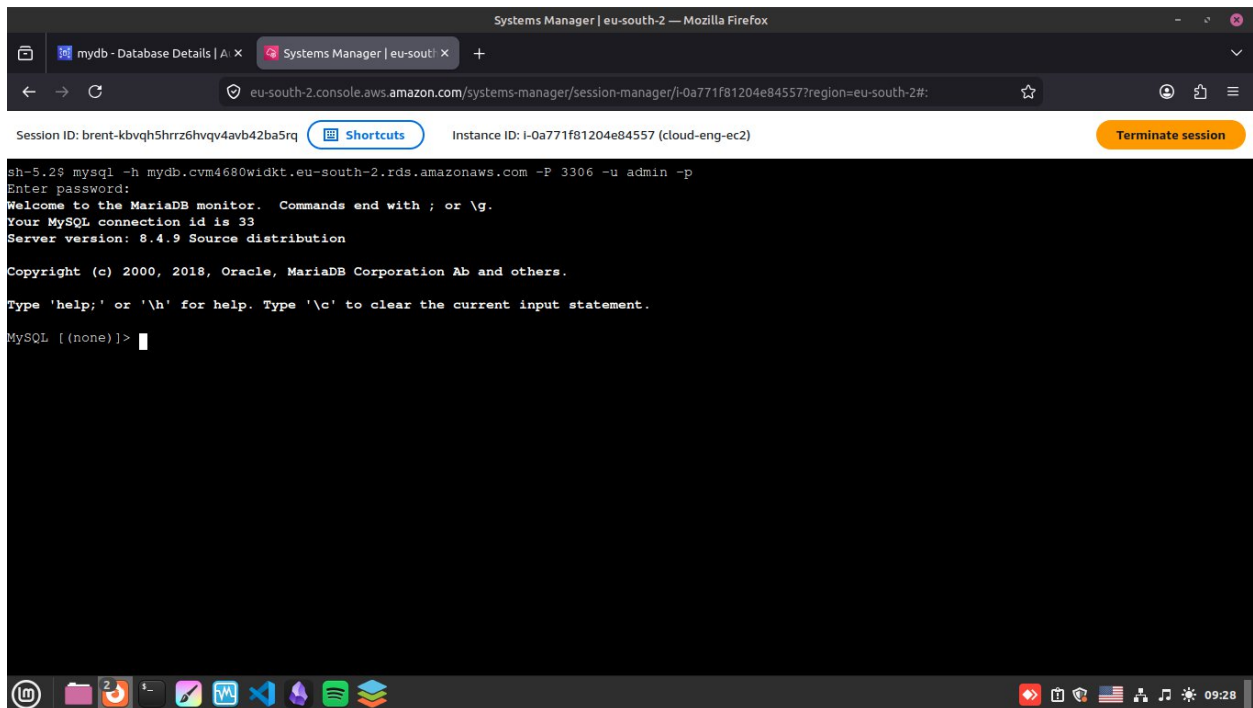
20:17

15. TESTING CONNECTIVITY

Now that the RDS DB is up and running, we connect the same way as before via SSM. We have to test connectivity to the DB from our EC2.

1. In your EC2 SSM shell, run 'sudo yum install mysql -y'. It depends on your instance if that's going to work or not. If it doesn't: 'yum search mysql' or 'yum search mariadb' and install the appropriate client. Mine turned out to be 'sudo yum install maria1011-client-utils -y'.
2. Once this is finished installing: 'mysql -h [your RDS DNS name] -u [your username from earlier] -p'. You'll be prompted for your password from earlier.

Did it connect? Congratulations, your SG-to-SG chain and NAT GW is all in order. If it doesn't connect, it is likely a SG problem. Make sure your DB SG is pointing to your App SG as a source, and not a CIDR block.



The screenshot shows a terminal window titled 'Systems Manager | eu-south-2' in a Mozilla Firefox browser. The address bar shows the URL: eu-south-2.console.aws.amazon.com/systems-manager/session-manager/i-0a771f81204e84557?region=eu-south-2#. The terminal output shows the command 'sh-5.29 mysql -h mydb.cvm4680widkt.eu-south-2.rds.amazonaws.com -P 3306 -u admin -p' being executed. The user enters a password, and the terminal displays: 'Welcome to the MariaDB monitor. Commands end with ; or \g. Your MySQL connection id is 33. Server version: 8.4.9 Source distribution. Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others. Type 'help;' or '\h' for help. Type '\c' to clear the current input statement. MySQL [(none)]>'. The terminal window is part of a desktop environment with various application icons at the bottom.

16. PLACEHOLDER APP SET UP

I'm not a web developer, you don't have to be either. We're going to set up a simple placeholder app that shows our directory listing when we hit the ALB just to make sure our infrastructure all connects. That's what we're here for.

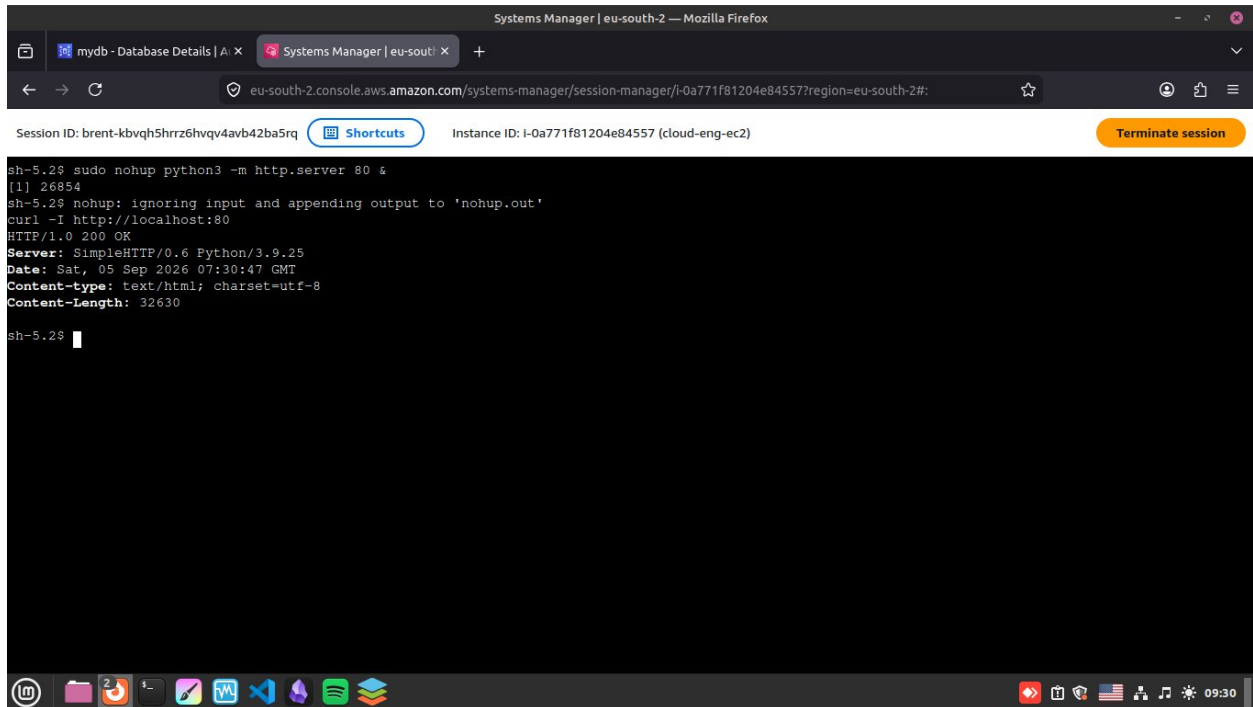
1. In your EC2 instance, exit your MySQL DB by typing 'exit'.
2. Run 'sudo nohup python3 -m http.server 80 &'

- This sets up a simple python3 server on port 80

- '&' will background the process and 'nohup' keeps it alive after you disconnect from SSM.

- Make sure it's listening with 'curl -I http://localhost:80'. You're looking for a 200 OK response in the header

3. Now that we achieved a 200 OK, we should lock down the app to only accept traffic from the ALB. In the App SG, change or make sure that the source is the ALB SG. This means we can only ever access the app via the load balancer and not directly from our IP nor anywhere on the internet.



```
sh-5.2$ sudo nohup python3 -m http.server 80 &
[1] 26854
sh-5.2$ nohup: ignoring input and appending output to 'nohup.out'
curl -I http://localhost:80
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.9.25
Date: Sat, 05 Sep 2026 07:30:47 GMT
Content-type: text/html; charset=utf-8
Content-Length: 32630
sh-5.2$
```

17. CREATE AN ALB AND ALB TG

For our final hurrah, we have to make a load balancer and target group for our project. Target Groups are used by your ELB/ALB to route incoming traffic. Load balancers are used to distribute incoming traffic across your instance(s) so as not to overload them.

CREATE ALB TG

1. In the EC2 Console, under 'Load Balancers', select Target Groups.
2. Create a TG. Type should be 'Instance', Protocol is HTTP on Port 80 (where we are serving our placeholder app). Select our VPC and the Health Check should be '/'. Give it a name and click next.
3. On the next screen, select the EC2 instance from the list, click 'Include as pending below' and create the TG. The TG will show as unhealthy or initializing for a little bit until the Health Check actually runs. It's normal.

CREATE ALB

1. Still in the EC2 Console, find 'Load Balancers' and create a LB. Select 'Application Load Balancer' and give it a name. Scheme is Internet-Facing and IP Type will be IPv4. Select our VPC and the 'Mappings' will be both public-1a and public-1b. ALBs require at least 2 AZs for High Availability.

2. In the SG section, select the ALB SG we made earlier and remove the default SG. Listeners and Routing should be HTTP on Port 80, forwarding to the Target Group we just made. Create it. It'll take a little to change status to 'Active'.

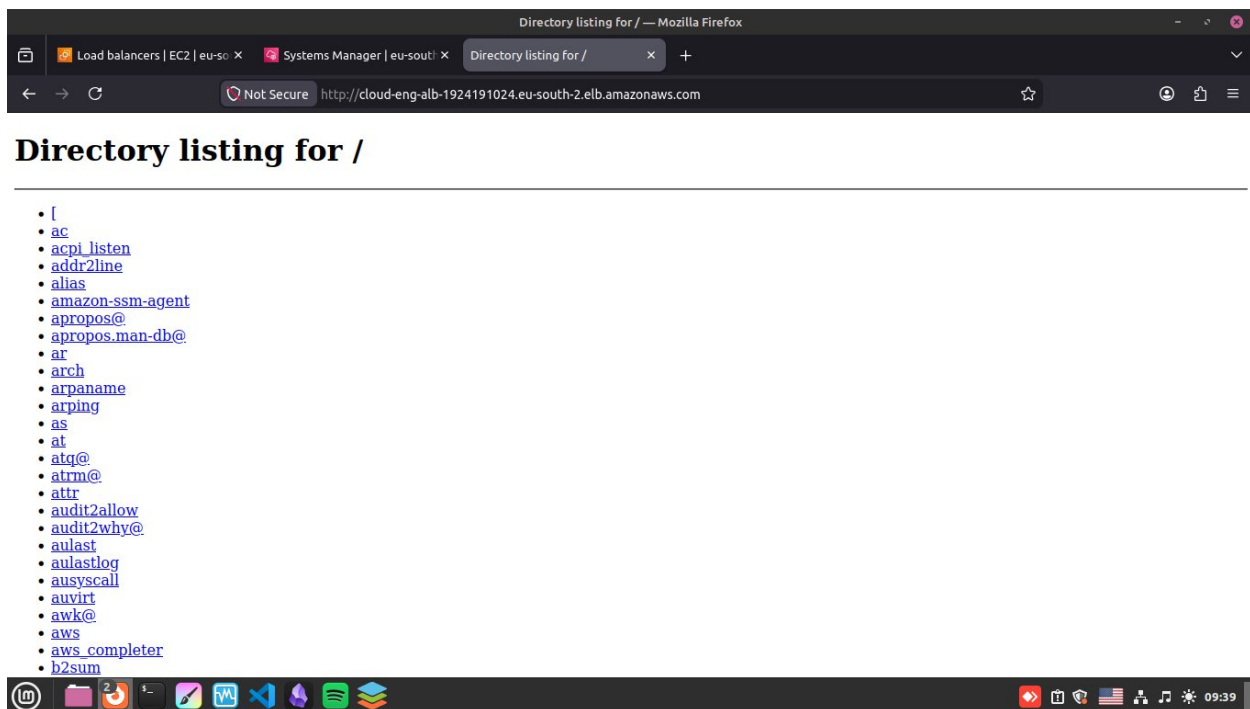
18. CROSS YOUR FINGERS

Once your ALB registers as 'Active', it's time to cross your fingers and hope it all works. We've been confirming connectivity and such ever since the beginning so it should, but more frustrating things have happened in the past, so, you know, cross 'em.

1. In the 'Load Balancers' section, click your ALB and scroll down a little bit.

2. Copy the DNS name for your ALB.

3. In your browser, type 'http:/' and paste your DNS name. You can also curl it in a separate terminal if you're cool. Did you get a directory listing? Great! Did you not? Not Great!



19. TEAR DOWN

We've got a few things running up a bill at the moment, so we need to delete them all before Bezos buys another house on our dime. Go to the respective sections and delete them:

1. NAT GW
2. Elastic IP
3. EC2 Instance
4. RDS DB (uncheck the final DB snapshot and auto backups)
5. ALB (and the TG. It's free but delete it.)